

Unsupervised Machine Learning

IBM Skills Network

Project Report

By:
Than Phuc Hau

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
Main Objective.....	3
Brief Description.....	3
Data cleaning & Feature engineering.....	4
Check for any duplicates.....	4
Check for missing value and datatype.....	4
Drop Unused Column.....	5
Encode Column.....	5
Check Skewness.....	6
Data correlation.....	7
Unsupervised Models.....	7
K-Means Clustering.....	7
Agglomerative Clustering.....	8
DBSCAN.....	9
Recommended Model.....	10
Key Findings and Insights.....	10
Suggestions.....	11

Main Objective

As living standards continue to rise, businesses are paying more attention to understanding their customers' needs and behaviors. Thanks to advances in technology and data analytics, large customer datasets are now easier to access and analyze. However, many companies still struggle to make sense of this data, which makes it difficult to identify customer groups and design effective marketing strategies.

This project focuses on analyzing customer information to predict and group customers into segments, helping businesses make more personalized and data-driven decisions.

Brief Description

The data set has many useful information that would help us identify and classify our target. Shown below is a snippet of `data.head()` which shows us a brief about the values of our columns. The data set has **200 examples**, and we have 5 features including CustomerID column. The data represents information about customers. The features are as follows:

- **CustomerID**
- **Gender**
- **Age**
- **Annual Income (k\$)**
- **Spending Score (1-100)**

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

In the next sections, we shall explore more information about the underlying patterns in this data set.

Data cleaning & Feature engineering

We want to first examine our data set and check for any missing values. After that, we will be checking for duplicates by using the 'CustomerID' column to check – for it being unique for each customer. We would use `data.info()` to check for missing values. As shown below, we can see that there are no missing values as we have 200 non-null out of 200 entries and there are no missing values.

Check for any duplicates

```
data.CustomerID.is_unique  
  
True
```

There is no duplicated value.

Check for missing value and datatype

```
data.info()  
  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 200 entries, 0 to 199  
Data columns (total 5 columns):  
#   Column                                Non-Null Count  Dtype  
---  -  
0   CustomerID                            200 non-null    int64  
1   Gender                                200 non-null    object  
2   Age                                    200 non-null    int64  
3   Annual Income (k$)                    200 non-null    int64  
4   Spending Score (1-100)                 200 non-null    int64  
dtypes: int64(4), object(1)  
memory usage: 7.9+ KB
```

There is no missing value in these columns, so we will bypass this step.

Drop Unused Column

We will drop CustomerID column as there is no use of it.

```
data.drop('CustomerID', axis = 1, inplace = True)
data.describe()
```

	Age	Annual Income (k\$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000
mean	38.850000	60.560000	50.200000
std	13.969007	26.264721	25.823522
min	18.000000	15.000000	1.000000
25%	28.750000	41.500000	34.750000
50%	36.000000	61.500000	50.000000
75%	49.000000	78.000000	73.000000
max	70.000000	137.000000	99.000000

Encode Column

We will encode Gender column to 0 and 1 to evaluate and train.

```
data["Gender"] = data["Gender"].map({"Male": 1, "Female": 0})
data
```

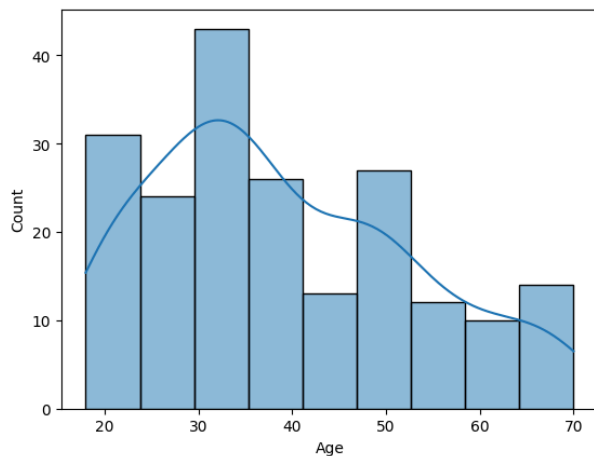
	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	19	15	39
1	1	21	15	81
2	0	20	16	6
3	0	23	16	77
4	0	31	17	40
...	...	...	...	...
195	0	35	120	79
196	0	45	126	28
197	1	32	126	74
198	1	32	137	18
199	1	30	137	83
200 rows x 4 columns				

Check Skewness

We want to check for skewness values for Gender, Age, Annual Income and Spending Score

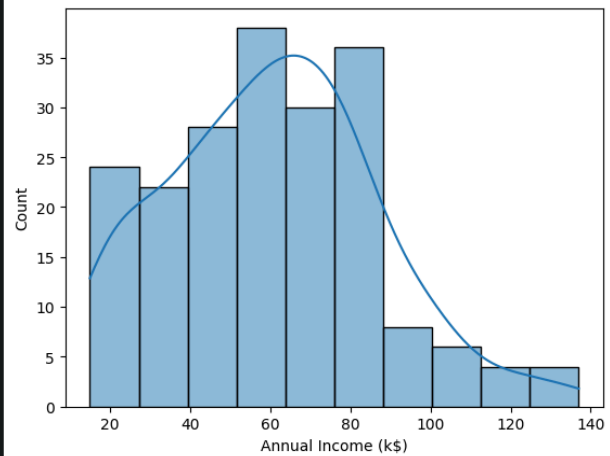
```
print('Skewness = ', data['Age'].skew())
sns.histplot(data['Age'], kde = True)
plt.show()
```

Skewness = 0.48556885096681657



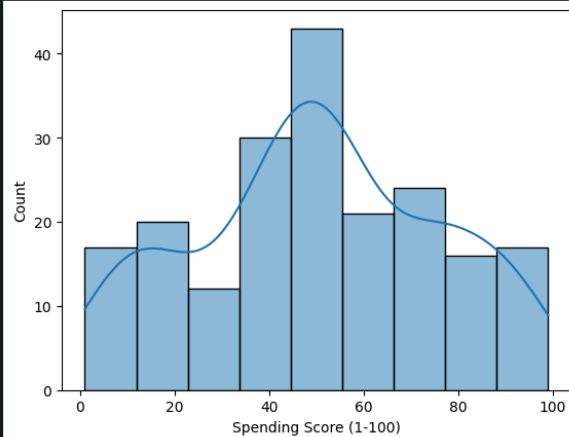
```
print('Skewness = ', data['Annual Income (k$)'].skew())
sns.histplot(data['Annual Income (k$)'], kde = True)
plt.show()
```

Skewness = 0.3218425498619055



```
print('Skewness = ', data['Spending Score (1-100)'].skew())
sns.histplot(data['Spending Score (1-100)'], kde = True)
plt.show()
```

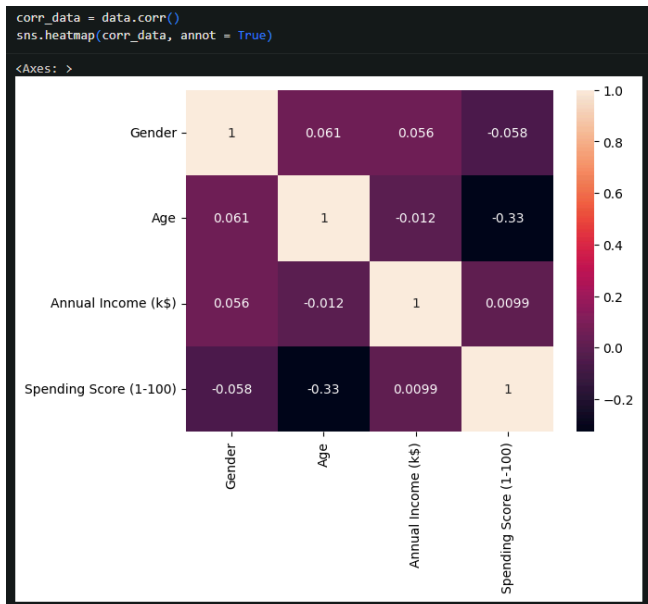
Skewness = -0.047220201374263374



As we can see from the histplots above, the columns Age, Annual Income and Spending Score all have reasonable skewness, so we do not need to apply transform for them.

Data correlation

We will visualize correlation between our features in order to have a basic idea of which of them correlate with which of the other features. As you can see below, there are the most correlation features printed together on each row using the `idxmax()` function.



```
for x in range(len(data.columns)):
    corr_data.iloc[x,x] = 0
corr_data.idxmax()
```

```
0
Gender      Age
Age         Gender
Annual Income (k$)  Gender
Spending Score (1-100)  Annual Income (k$)
dtype: object
```

Unsupervised Models

We will be using **K-Means**, **AgglomerativeClustering**, and **DBSCAN** as our clustering models.

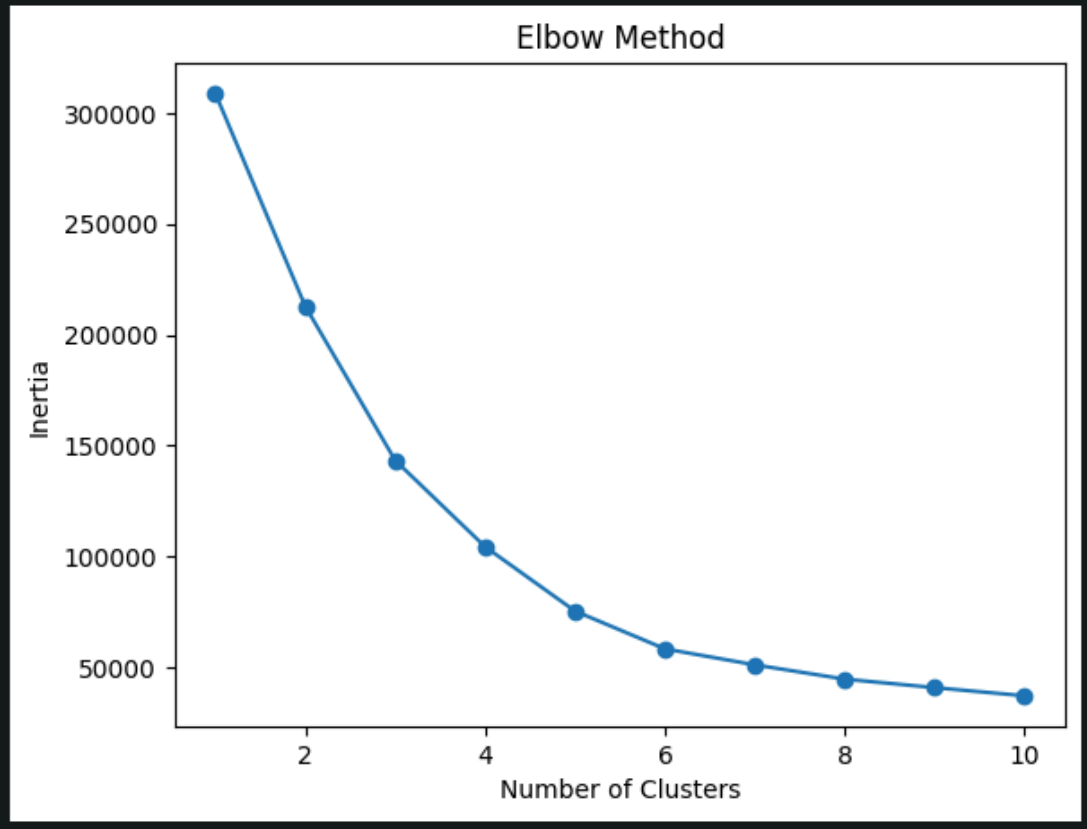
K-Means Clustering

We will first examine the K-Means model as a base-line. We will be using the MiniBatch version as it is fast. We will be using the elbow method to determine which k is best using an inertia plot. However, the plot did not give us any idea of the best value for k.

```
from sklearn.cluster import KMeans

inertia_values = []
for n_clusters in range(1, 11):
    kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
    kmeans.fit(data)
    inertia_values.append(kmeans.inertia_)
```

```
plt.plot(range(1, 11), inertia_values, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.show()
```



Agglomerative Clustering

We will then train our AgglomerativeClustering model and try to visualize the dendrogram of the model using **ward linkage**.

```

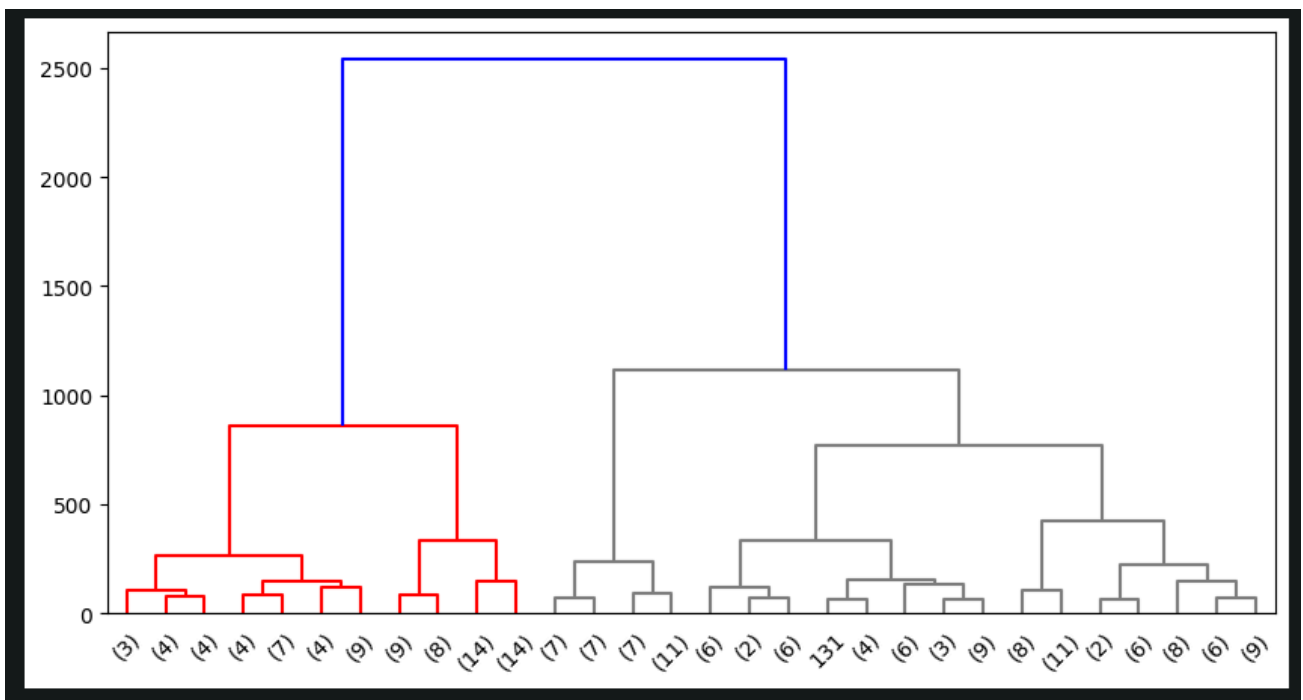
from scipy.cluster import hierarchy
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt

ag = AgglomerativeClustering(n_clusters = 10, linkage = 'ward', compute_full_tree = True)
ag.fit(data)

Z = hierarchy.linkage(ag.children_, method = 'ward')
fig, ax = plt.subplots(figsize = (10,5))
hierarchy.set_link_color_palette(['red','gray'])
den = hierarchy.dendrogram (Z, orientation = 'top',
                             p = 30, truncate_mode = 'lastp',
                             show_leaf_counts = True, ax = ax,
                             above_threshold_color = 'blue')

plt.show()

```



DBSCAN

For our last model we will be using DBSCAN to try to not specify the number of clusters and see if we can better cluster our data. However, DBSCAN requires two parameters and finding appropriate values of epsilon and n_{clu} can be difficult. We have used a $\epsilon=0.5$ and $min_samples=20$ for our DBSCAN model which lead to group our data into 1 clusters.

```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps = 0.5, min_samples = 20)
db = db.fit(data)
clusters = db.labels_
pd.DataFrame(clusters).nunique()
```

```
0
0 1
dtype: int64
```

Recommended Model

Without a doubt, each of our models did its best to try to cluster our data given the specific hyperparameters of each of them. However, we can see that a DBSCAN can be hard to train and find the right hyperparameters. Therefore, we recommend the usage of **AgglomerativeClustering** as the recommended model in order to help visualize our customer segmentation more. This would help us negotiate with decision makers and lead to moderate results.

Key Findings and Insights

Our analysis showed that using the **elbow method** on an inertia plot for the **K-Means** algorithm did not always provide a clear optimal number of clusters. In this case, the curve lacked a distinct “elbow” point, making it difficult to determine the best cluster count. On the other hand, the **Agglomerative Clustering** model offered a more intuitive visualization through **dendrograms**, which proved useful in customer segmentation. This approach allowed us to better identify and understand different customer groups based on age, gender, annual income, and spending score.

Suggestions

There is still room for improvement in all models. One possible enhancement is to apply **GridSearchCV** to optimize model hyperparameters and improve clustering performance. Additionally, experimenting with other evaluation metrics could provide a more comprehensive understanding of cluster quality. The **Mean-Shift** algorithm might also be a suitable alternative, as it eliminates the need to predefine the number of clusters. Finally, incorporating more customer attributes — such as shopping frequency, preferred product categories, or online activity — and applying **PCA** or **LDA** could help uncover deeper relationships within the data.